

CICLE FORMATIU · DESENVOLUPAMENT D'APLICACIONS WEB · MÒDUL ACCÉS A
DADES

Examen de Verificació Django MVT 2025–2026

ALUMNE/A

Espinal

Nota: Les preguntes d'aquest examen han estat generades a partir dels **repositoris lliurats per l'alumne/a** a les activitats corresponents de Django MVT. El codi mostrat prové directament de la implementació pràctica del/de la alumne/a.

BLOC 1

FORM

1

models.py — Classe interna RoL al model User

TextChoices i camps amb choices · Nivell bàsic

5 pts

Pregunta 1 de 12 · Total: 100 pts

El teu model User defineix una classe interna per gestionar els rols permesos. Observa el codi següent:

```
from django.db import models

class User(models.Model):
    class RoL(models.TextChoices):
        PROFESSOR = "Professor"
        ALUMNE = "Alumne"

    nom = models.CharField(max_length=50)
    cognom = models.CharField(max_length=50)
    edat = models.IntegerField()
    correu = models.EmailField(max_length=150)
    aula = models.CharField(max_length=5)
    rol = models.CharField(max_length=30, choices=RoL.choices)
```

a) Què és models.TextChoices i per quin motiu has decidit usar-lo per al camp rol en lloc d'un CharField simple sense choices?

2 pts

b) El valor real que s'emmagatzema a la base de dades quan es tria RoL.PROFESSOR és "Professor" (la cadena de text) o és "PROFESSOR" (el nom de la constant)? Explica per quin motiu.

2 pts

c) Quina diferència hi hauria si haguessis posat max_length=10 en lloc de max_length=30 al camp rol?

1 pt

RESPOSTA

Escriu la teva resposta aquí...

FORM

2

forms.py — UserForm i la declaració de camps

ModelForm i atribut fields · Nivell bàsic

5 pts

Pregunta 2 de 12 · Total: 100 pts

El teu formulari hereda de `ModelForm` i especifica quins camps s'han d'incloure. Observa la implementació:

```
from django.forms import ModelForm
from .models import User

class UserForm(ModelForm):
    class Meta:
        model = User
        fields = "__all__"
```

- a) Quan poses `fields = "__all__"`, quins camps del model `User` apareixeran al formulari renderitzat al navegador? Inclourà el camp `id`? 2 pts
- b) Quina diferència pràctica hi hauria si en lloc de `"__all__"` haguessis escrit `fields = ['nom', 'cognom', 'edat', 'correu', 'aula', 'rol']`? 2 pts
- c) Hi ha alguna situació en un projecte real on preferiries la llista explícita a `"__all__"`? Per quin motiu? 1 pt

RESPOSTA

Escriu la teva resposta aquí...

FORM

3

views.py — Flux GET/POST de la vista user_form

Creació d'un nou registre amb formulari · Nivell mitjà

8 pts

Pregunta 3 de 12 · Total: 100 pts

La teva vista user_form gestiona tant la petició inicial (mostrar el formulari buit) com l'enviament de dades (desar el nou usuari). Observa el codi:

```
def user_form(request):  
    form = UserForm()  
  
    if request.method == "POST":  
        form = UserForm(request.POST)  
  
        if form.is_valid():  
            form.save()  
            return redirect("get_all_users")  
  
    formulari = {"formulari": form}  
    return render(request, "formulari.html", formulari)
```

- a) Explica pas a pas el flux d'execució quan un usuari entra per primera vegada a la URL d'aquest formulari (petició GET): quina línia s'executa i quina no, i per quin motiu.
- b) Quan la petició és POST però el formulari no és vàlid (per exemple, el camp edat conté text), quin és el comportament de la vista? On acaba l'execució i què veu l'usuari?
- c) Per quin motiu has fet servir `redirect("get_all_users")` en lloc de `render(request, "users.html", ...)` després de desar?

3 pts

3 pts

2 pts

RESPOSTA

Escriu la teva resposta aquí...

FORM

4

8 pts

views.py — Paràmetre instance= a la vista update_user_form

Pregunta 4 de 12 · Total: 100 pts

Actualització de registre existent · Nivell mitjà

La teva funció `update_user_form` utilitza el paràmetre `instance=` en dos llocs: en la creació del formulari per al GET i en la recollida de dades del POST. Observa el codi:

```
def update_user_form(request, pk):
    user_id = User.objects.get(id=pk)
    form = UserForm(instance=user_id)

    if request.method == "POST":
        form = UserForm(request.POST, instance=user_id)

        if form.is_valid():
            form.save()
            return redirect("get_all_users")

    formulari = {"formulari": form}
    return render(request, "formulari.html", formulari)
```

a) Quin és el propòsit de `instance=user_id` en la línia del GET (`form = UserForm(instance=user_id)`)? Quin efecte visual té al formulari renderitzat?

3 pts

b) Quin seria l'efecte concret sobre la base de dades si al POST haguessis escrit `form = UserForm(request.POST)` sense el `instance=user_id`?

3 pts

c) Per quin motiu reutilitzes el mateix template `formulari.html` tant per crear com per actualitzar? Quins avantatges té aquesta decisió?

2 pts

RESPOSTA

Escriu la teva resposta aquí...

TAGS

5

5 pts

views.py — Transformació del paràmetre a getUsersByRole

Pregunta 5 de 12 · Total: 100 pts

Filtratge per rol amb normalització · Nivell bàsic

A la vista `getUsersByRole` de la branca **tags**, apliques una transformació al paràmetre `rol` rebut de la URL abans de fer la consulta a la base de dades:

```
def getUsersByRole(request, rol):  
    persons = Person.objects.filter(rol=rol.upper())  
    llista_persons = {"llista_persons": persons}  
    return render(request, "persons.html", llista_persons)
```

Al template `index.html` d'aquest repositori, els enllaços de filtratge passen el rol en minúscules:

```
<li><a href="{% url 'get_all_users_by_role' 'alumne' %}">Tots els persons alumnes</a>  
<li><a href="{% url 'get_all_users_by_role' 'professor' %}">Tots els persons professor</li>
```

a) Per quin motiu has afegit `.upper()` al paràmetre `rol` abans de fer el `filter`? Quina relació té amb la forma en què s'han guardat els rols a la base de dades?

2 pts

b) Quina diferència hi ha entre la forma en què el repositori *form* gestiona els rols (amb `TextChoices`) i la forma en què ho fa el repositori *tags* (sense `TextChoices`)? Quin efecte té sobre la consulta `filter`?

2 pts

c) Si un usuari accedís manualment a la URL `/users/AlUmNe/`, la consulta retornaria resultats? Per quin motiu?

1 pt

RESPOSTA

Escriu la teva resposta aquí...

FORM

6

urls.py — Converters de tipus als patrons d'URL

Tipus de converters i ordre de patrons · Nivell mitjà

8 pts

Pregunta 6 de 12 · Total: 100 pts

Al fitxer `urls.py` de la branca **form** tens definits patrons amb converters de tipus diferent per a cada URL:

```
urlpatterns = [
    path("getAllUsers/", views.get_all_users, name="get_all_users"),
    path("users/<str:rol>/", views.get_users_by_role, name="get_all_users_by_r"),
    path("user/<int:pk>/", views.get_user_by_id, name="get_one_user"),
    path("formulari/", views.user_form, name="formulari"),
    path("update_user/<int:pk>", views.update_user_form, name="update_user"),
    path("delete_user/<int:pk>", views.delete_user, name="delete_user"),
]
```

a) Quina diferència hi ha entre el converter `<str:rol>` i el converter `<int:pk>`? Per quin motiu cada URL utilitza un tipus diferent?

3 pts

b) Si haguessis posat `<str:pk>` en lloc de `<int:pk>` a la URL de `get_user_by_id`, el codi de la vista hauria funcionat correctament amb `User.objects.get(id=pk)`? Explica per quin motiu.

3 pts

c) Per quin motiu el patró `users/<str:rol>/` no interfereix amb cap dels altres patrons de la llista?

2 pts

RESPOSTA

Escriu la teva resposta aquí...